

Lập trình sử dụng API -
Các cửa sổ điều khiển

Nguyễn Thu Hường

Khoa CNTT – Đại học GTVT

Nội dung của chương

- Lớp nút (button class)
  - Các nút ấn (push buttons)
  - Các hộp chọn (check boxes)
  - Các hộp radio (radio buttons)
  - Các hộp nhóm (group boxes)
- Lớp tĩnh (static class)
- Lớp soạn thảo (edit class)
- Lớp hộp danh sách (listbox class)
- Lớp thanh cuốn (scroll bar class)
- Hệ thống màu, các nút tự vẽ

Cửa sổ con

- Là cửa sổ mà có kiểu cửa sổ là WS_CHILD và nó bị giới hạn trong miền thao tác của cửa sổ cha
- Một ứng dụng thông thường sử dụng các cửa sổ con để chia miền thao tác của cửa sổ cha thành các vùng nhỏ hơn có các chức năng khác nhau
- Một cửa sổ con bao giờ cũng phải có cửa sổ cha, cha của nó có thể là:
  - Một cửa sổ thông thường
  - Một cửa sổ pop-up (hộp hội thoại)
  - Thậm chí là một cửa sổ con khác
- Nếu ta dùng kiểu cửa sổ WS_CHILD trong CreateWindow nhưng lại không cho biết cửa sổ cha của nó, khi đó Windows sẽ **không tạo** ra cửa sổ nào cả

Cửa sổ con

- Windows luôn định vị cửa sổ con so với góc trên bên trái của miền thao tác của cửa sổ cha
- Không có phần nào của cửa sổ con lại có thể xuất hiện bên ngoài các viền của cửa sổ cha
- Windows sẽ thực hiện công việc cắt xén cửa sổ con; có nghĩa là, phần nằm ngoài miền thao tác của cửa sổ cha sẽ không được hiển thị, điều này xảy ra khi:
  - Ứng dụng tạo ra một cửa sổ con mà có kích thước lớn hơn cửa sổ cha
  - Ứng dụng định vị cửa sổ con mà một phần nào đó của cửa sổ con nằm ngoài các viền của cửa sổ cha

Cửa sổ con

- Cửa sổ con xử lý các thông điệp chuột và bàn phím và nó báo cho cửa sổ cha biết mỗi khi trạng thái của cửa sổ con bị thay đổi
- Như vậy, cửa sổ con trở thành thiết bị đầu vào ở tầng cao đối với cửa sổ cha
- Cửa sổ con có chứa:
  - Một số chức năng cụ thể có liên quan đến diện mạo bề ngoài của nó trên màn hình
  - Phản ứng của nó với dữ liệu nhập vào từ người sử dụng
  - Cách thức thông báo cho các cửa sổ khác biết mỗi khi một sự kiện đầu vào quan trọng vừa xuất hiện

Cửa sổ con

- Mặc dù có thể tạo cho mình các cửa sổ điều khiển riêng, song chúng ta cũng có thể dùng một số lớp cửa sổ (và các thủ tục cửa sổ) đã có sẵn
  - Các nút (tên lớp: button)
  - Các hộp soạn thảo (tên lớp: edit)
  - Các hộp danh sách (tên lớp: listbox)
  - Các hộp kết hợp (tên lớp: combobox)
  - Các văn bản tĩnh (tên lớp: static)
  - Các thanh cuốn (tên lớp: scrollbar)

Cửa sổ con

- Các cửa sổ điều khiển được sử dụng thường xuyên trong các hộp hội thoại
- Tuy nhiên, chúng ta cũng có thể sử dụng các cửa sổ điều khiển đã có sẵn trên bề mặt miền thao tác của một cửa sổ thông thường
  - Tạo cửa sổ con nhờ hàm CreateWindow
  - Điều chỉnh vị trí và kích thước của chúng nhờ hàm MoveWindow
- Liên lạc giữa cửa sổ cha và cửa sổ con
  - Cửa sổ cha gửi thông điệp tới cửa sổ con
  - Cửa sổ con gửi thông điệp tới cửa sổ cha

Cửa sổ con

- Tạo cửa sổ con (cửa sổ điều khiển):
  - Dùng từ khóa #define để định nghĩa định danh chữ (số) cho của sổ.
  - Khai báo cửa sổ có kiểu HWND
  - Dùng hàm CreateWindow để tạo cửa sổ điều khiển
- Ví dụ: Tạo cửa sổ edit để nhập liệu

```
#define ID_EDIT2
HWND hwndEdit;
```

```
hwndEdit = CreateWindow(TEXT("edit"), //Tên lớp cửa sổ
    TEXT(""), //Nội dung hiển thị trên cửa sổ
    WS_CHILD | WS_VISIBLE|WS_BORDER, //Kiểu cửa sổ
    210, 100, 200, 50, //Vị trí và kích thước cửa sổ
    hwnd, //Hadle cửa sổ cha
    (HMENU)ID_EDIT, NULL, NULL); //ID chữ (số) đại diện cho nó
```

Cửa sổ con

- Việc sử dụng các cửa sổ điều khiển trên bề mặt của 1 cửa sổ thông thường sẽ đòi hỏi ứng dụng phải thực hiện thêm một số nhiệm vụ so với việc sử dụng chúng trong các hộp hội thoại
- Cụ thể, đối với các cửa sổ điều khiển con mà được tạo trên bề mặt của cửa sổ thông thường:
  - Không thể sử dụng phím Tab hoặc các phím di chuyển con chạy để di chuyển tiêu điểm nhập từ một điều khiển sang điều khiển khác
  - Một cửa sổ điều khiển có thể nhận được tiêu điểm nhập, nhưng sau đó nó sẽ không tự do nhả tiêu điểm nhập trở lại cho cửa sổ cha

Cửa sổ con giao tiếp với cửa sổ cha

- Khi chúng ta nháy chuột vào một cửa sổ đk, cửa sổ điều khiển sẽ gửi thông điệp WM_COMMAND tới cửa sổ cha của nó
- Trong WM_COMMAND Sử dụng các giá trị wParam và lParam với các ý nghĩa của chúng:

LOWORD (wParam)	ID của cửa sổ con
HIWORD (wParam)	Mã khai báo
lParam	Handle của cửa sổ con

Mã khai báo

Tên mã khai báo của các nút	Ý nghĩa
BN_CLICKED	Người sử dụng nháy chuột vào nút
BN_PAINT	Nút cần được vẽ
BN_HILITE hoặc BN_PUSHED	Nút được nhấn
BN_UNHILITE hoặc BN_UNPUSHED	Nút được nhả
BN_DISABLE	Nút bị disabled
BN_DOUBLECLICKED hoặc BN_DBLCLK	Người sử dụng nháy kép chuột vào nút
BN_SETFOCUS	Nút nhận tiêu điểm nhập
BN_KILLFOCUS	Nút mất tiêu điểm nhập

Cửa sổ cha giao tiếp với cửa sổ con



- Thủ tục cửa sổ cha cũng có thể gửi các thông điệp đến cửa sổ điều khiển, bao gồm:
  - Những thông điệp bắt đầu bằng tiền tố WM
  - Ngoài ra còn có 8 thông điệp với tiền tố BM dành riêng cho các nút
    - BM_GETCHECK, BM_SETCHECK, BM_GETSTATE, BM_SETSTATE
    - BM_SETSTYLE, BM_CLICK, BM_GETIMAGE, BM_SETIMAGE
    - Dùng hàm SendMessage để gửi các thông điệp này đi

Lấy định danh và handle

- Nếu biết handle của cửa sổ con, ta có thể lấy được định danh của nó:

```
id = GetWindowLong (hwndChild, GWL_ID) ;
id = GetDlgCtrlID (hwndChild) ;
```

Chú ý: Không cần đến thông tin về cửa sổ cha!!!

- Khi biết được định danh của cửa sổ con và handle của cửa sổ cha, ta có thể biết được handle của cửa sổ con:

```
hwndChild = GetDlgItem (hwndParent, id) ;
```

Các nút ấn

- Là một hình chữ nhật, bên trong có chứa nhãn của nút
- Nhãn được hiển thị ở trung tâm của hình chữ nhật
- Các nút ấn được sử dụng để kích hoạt một hành động tức thời mà không lưu lại bất cứ một kiểu biểu thị bật/tắt nào
- Có 2 kiểu nút ấn, chúng có kiểu cửa sổ là
  - BS_PUSHBUTTON
  - BS_DEFPUSHBUTTON

Các nút ấn

- Việc nhấn nút bằng chuột sẽ khiến nút này vẽ lại nó bằng cách sử dụng sự đánh bóng 3D để biểu thị nó đang được nhấn
- Khi nhả nút chuột:
  - Hình dáng ban đầu của nút được khôi phục
  - WM_COMMAND được gửi tới cửa sổ cha với mã khai báo là BN_CLICKED
- Cũng giống như các kiểu nút khác, khi nút ấn sở hữu tiêu điểm nhập:
  - Có một đường gạch ngang bao quanh nhãn của nút
  - Việc nhấn và nhả thanh dài cũng có tác dụng tương tự như việc nhấn và nhả chuột

Các nút ấn

- Có thể làm giả động tác nháy chuột lên nút ấn:
  - SendMessage (hwndButton, BM_SETSTATE, 1, 0) ;
- Câu lệnh sau sẽ khiến nút trở lại trạng thái bình thường:
  - SendMessage (hwndButton, BM_SETSTATE, 0, 0) ;
- Ta cũng có thể gửi thông điệp BM_GETSTATE tới một nút ấn, cửa sổ điều khiển sẽ trả ra:
  - TRUE - nếu nút đang bị nhấn
  - FALSE - nếu nút không bị nhấn

Các hộp chọn

- Là một hình vuông kèm theo nhãn
- Nhãn thường xuất hiện ở bên phải của hộp chọn
- Nếu ta thêm vào kiểu BS_LEFTTEXT khi tạo nút:
  - Nhãn sẽ xuất hiện ở bên trái hộp chọn
  - Có thể kết hợp kiểu này với BS_RIGHT để chỉnh nhãn theo mép phải
- Hộp chọn cũng giống như một công tắc:
  - Nháy chuột một lần vào hộp chọn: sẽ khiến dấu chọn xuất hiện
  - Nháy chuột một lần nữa: sẽ làm dấu chọn biến mất
- Cho phép người dùng chọn các tùy chọn

Các hộp chọn

- Hai kiểu thông dụng nhất của hộp chọn:
  - BS_CHECKBOX
  - BS_AUTOCHECKBOX
- Hộp chọn kiểu BS_CHECKBOX
  - Ta phải tự thiết lập dấu chọn (khi người dùng nhấn chuột) bằng cách gửi tới hộp chọn thông điệp BM_SETCHECK

```
SendMessage ((HWND) lParam, BM_SETCHECK, (WPARAM)
!SendMessage ((HWND) lParam, BM_GETCHECK, 0, 0), 0) ;
```

  - Ta có thể biết trạng thái hiện thời của hộp chọn bằng việc gửi tới hộp chọn thông điệp BM_GETCHECK

Các hộp chọn

- Hộp chọn kiểu BS_AUTOCHECKBOX
  - Hộp chọn tự nó thực hiện công việc bật/tắt dấu chọn
  - Khi cần biết trạng thái hiện thời của nút, ta hãy gửi cho nó thông điệp BM_GETCHECK:
    - iCheck = (int) SendMessage (hwndButton, BM_GETCHECK, 0, 0) ;
    - Giá trị của iCheck:
      - TRUE - nút được chọn
      - FALSE - nút không được chọn

Các hộp chọn

- Còn hai kiểu hộp chọn khác là:
  - BS_3STATE
  - BS_AUTO3STATE
- Các kiểu hộp chọn này có thể hiển thị thêm trạng thái thứ 3:
  - Hiển thị một dấu chọn màu xám bên trong hộp chọn
  - Được thiết lập bằng cách gửi cho hộp chọn thông điệp BM_SETCHECK với wParam bằng 2
  - Biểu thị sự lựa chọn là mập mờ hoặc không thích hợp

Các hộp chọn

- Việc nháy chuột vào bất cứ đâu bên trong hình chữ nhật của hộp chọn sẽ khiến thông điệp WM_COMMAND được gửi tới cửa sổ cha
- Kích thước cực tiểu của hộp chọn:
  - Chiều cao cực tiểu = độ cao của 1 ký tự
  - Chiều rộng cực tiểu = số ký tự có trong nhãn + 2

Các nút radio

- Được đặt tên theo một hàng nút mà đã từng xuất hiện phổ biến trên các máy thu thanh (radio) của ôtô
- Trong các hộp hội thoại, các nhóm nút radio theo thường lệ được sử dụng để biểu thị các lựa chọn loại trừ nhau
- Hoạt động không giống như các nút bật tắt - khi nháy chuột lần thứ 2 vào một nút radio thì trạng thái của nó vẫn không thay đổi
- Nút radio có kiểu cửa sổ:
  - BS_RADIOBUTTON
  - BS_AUTORADIOBUTTON

Các nút kiểu BS_RADIOBUTTON

- Khi nhận thông điệp WM_COMMAND từ một nút radio:
  - Ta cần phải hiển thị dấu chọn của nó bằng cách gửi tới nó thông điệp BM_SETCHECK với wParam bằng 1:

```
SendMessage (hwndButton, BM_SETCHECK, 1, 0) ;
```

  - Đối với tất cả các nút radio khác trong cùng một nhóm, ta tắt các dấu chọn bằng cách gửi tới chúng thông điệp BM_SETCHECK với wParam bằng 0:

```
SendMessage (hwndButton, BM_SETCHECK, 0, 0) ;
```

Các hộp nhóm

- Là nút có kiểu BS_GROUPBOX
- Hơi khác thường so với các kiểu khác trong nhóm nút
  - Không xử lý các sự kiện chuột và bàn phím
  - Không gửi các thông điệp WM_COMMAND đến cửa sổ cha
- Là một đường bao hình chữ nhật có nhãn nằm ở đỉnh của nó
- Thường được sử dụng để nhóm các điều khiển khác

Nhãn của nút

- Vị trí hiển thị nhãn
  - Đối với một cửa sổ bình thường - được hiển thị trên thanh tiêu đề
  - Đối với một nút điều khiển - được hiển thị trên nút
- Thay đổi nhãn của nút (hoặc của bất kỳ một cửa sổ nào khác):

```
SetWindowText (hwnd, pszString) ;
```

- Lấy nhãn hiện thời của cửa sổ:

```
iLength = GetWindowText(hwnd, pszBuffer, iMaxLength) ;
```

Các nút visible và enabled

- Để nhận các dữ liệu từ chuột và bàn phím, một cửa sổ con đồng thời phải là visible và enabled
- Khi một cửa sổ con visible nhưng disabled, Windows sẽ hiển thị nhãn của cửa sổ bằng màu xám chứ không phải màu đen nữa

Các nút visible và enabled

- Nếu không chỉ định kiểu WS_VISIBLE khi tạo một cửa sổ con, ta phải gọi hàm ShowWindow để hiển thị cửa sổ:

```
ShowWindow(hwndChild, SW_SHOWNORMAL) ;
```

- Làm cho cửa sổ con enabled hoặc disabled

```
EnableWindow (hwndChild, fDspl) ;
```

  - fDspl == TRUE – cửa sổ enabled
  - fDspl == FALSE – cửa sổ disabled

- Xác định một cửa sổ con có đang là enabled hay không:

```
IsWindowEnabled (hwndChild) ;
```

Các nút và tiêu điểm nhập

- Các nút ấn, các hộp chọn, các nút radio và cả những nút tự vẽ sẽ nhận tiêu điểm nhập khi chúng được nháy bởi chuột
- Khi cửa sổ điều khiển nhận tiêu điểm nhập:
  - Cửa sổ cha sẽ mất tiêu điểm nhập
  - Tất cả các sự kiện bàn phím sẽ đến cửa sổ điều khiển chứ không còn đến cửa sổ cha nữa
  - Tuy nhiên, cửa sổ điều khiển chỉ phản ứng lại thanh dài, chức năng của nó giống như của chuột
- Nảy sinh một vấn đề: chương trình đã đánh mất quyền điều khiển các sự kiện bàn phím!!!

Các nút và tiêu điểm nhập

- Khi Windows chuyển tiêu điểm nhập từ một cửa sổ (như cửa sổ cha) sang cửa sổ khác (ví dụ cửa sổ điều khiển)
  - Đầu tiên, Windows gửi thông điệp WM_KILLFOCUS đến cửa sổ mà đang mất tiêu điểm nhập
    - wParam - handle của cửa sổ mà sẽ nhận tiêu điểm nhập
  - Sau đó, Windows gửi thông điệp WM_SETFOCUS cho cửa sổ sẽ nhận tiêu điểm nhập
    - wParam - handle của cửa sổ đang đánh mất tiêu điểm nhập
  - Trong cả 2 trường hợp, wParam có thể là NULL, nó cho biết không có cửa sổ nào đánh mất hoặc nhận tiêu điểm nhập cả

Các nút và tiêu điểm nhập

- Cửa sổ cha có thể ngăn cản con của nó (cửa sổ điều khiển) nhận tiêu điểm nhập:
  - **Cách 1:**

```
case WM_KILLFOCUS :
    for (i = 0 ; i < NUM ; i++)
        if (hwndChild [i] == (HWND) wParam)
        {
            SetFocus (hwnd) ;
            break ;
        }
    return 0 ;
```

  - **Cách 2:**

```
case WM_KILLFOCUS :
    if (hwnd == GetParent ((HWND) wParam))
        SetFocus (hwnd) ;
    return 0 ;
```

Các nút và tiêu điểm nhập

- Cả hai phương pháp trên đều có nhược điểm:
  - Nút không bao giờ nhận được tiêu điểm nhập
  - Cản trở nút phản ứng lại thanh dài
- Cách tiếp cận tốt hơn:
  - Cho phép nút nhận được tiêu điểm nhập
  - Đồng thời tạo điều kiện thuận lợi cho người sử dụng di chuyển từ nút này sang nút khác bằng cách dùng phím **Tab**

Lớp tĩnh (static)

- Tên lớp: static
- Cửa sổ tĩnh:
  - Không nhận các sự kiện chuột và bàn phím
  - Không gửi các thông điệp WM_COMMAND cho sổ cha

Lớp tĩnh

- Khi ta di chuyển hoặc nháy chuột trên cửa sổ tĩnh:
  - Cửa sổ này chặn thông điệp WM_NCHITTEST và trả ra cho Windows giá trị HTTRANSPARENT
  - Điều này khiến Windows gửi một thông điệp WM_NCHITTEST tương tự tới cửa sổ nằm dưới - thông thường đó là cửa sổ cha của nó
  - Cửa sổ cha thông thường lại truyền thông điệp này vào hàm DefWindowProc để thông điệp được chuyển đổi thành thông điệp chuột trong miền thao tác

Lớp tĩnh

- Sáu kiểu cửa sổ tĩnh đầu tiên:
  - Đơn thuần chỉ vẽ một hình chữ nhật hoặc một khung bên trong miền thao tác của cửa sổ con
  - Trường nhãn khi gọi hàm CreateWindow được bỏ qua đối với các kiểu này

SS_BLACKRECT	SS_BLACKFRAME
SS_GRAYRECT	SS_GRAYFRAME
SS_WHITERECT	SS_WHITEFRAME

Cửa sổ điều khiển tĩnh	Màu hệ thống
BLACK	COLOR_3DDKSHADOW
GRAY	COLOR_BTNSHADOW
WHITE	COLOR_BTNHIGHLIGHT

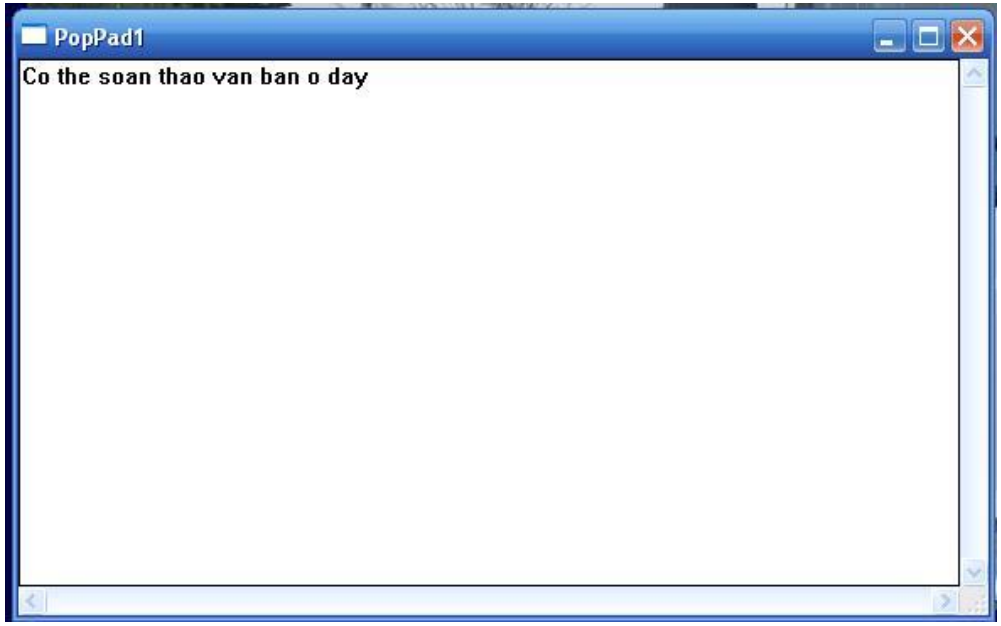
Lớp tĩnh

- Lớp tĩnh đồng thời cũng có 3 kiểu văn bản:
  - SS_LEFT
  - SS_RIGHT
  - SS_CENTER
- Văn bản
  - Được truyền vào thông qua tham số nhãn của hàm CreateWindow
  - Sau đó nó có thể được thay đổi thông qua hàm SetWindowText

Lớp soạn thảo

- Tên lớp soạn thảo - edit
- Hộp soạn thảo là 1 hình chữ nhật chứa văn bản có thể soạn thảo được
- Khi hộp soạn thảo có tiêu điểm nhập:
  - Có thể gõ văn bản
  - Di chuyển con chạy
  - Lựa chọn các mẩu văn bản bằng cách:
    - Sử dụng chuột
    - Hoặc sử dụng phím Shift kết hợp với một phím dịch chuyển con chạy
  - Xoá văn bản đã lựa chọn và đưa vào trong clipboard bằng việc nhấn Ctrl-X
  - Sao chép văn bản bằng việc nhấn Ctrl-C
  - Chèn văn bản vào từ clipboard bằng việc nhấn Ctrl-V

Chương trình POPPAD1



- POPPAD1 là trình soạn thảo cho phép soạn thảo nhiều dòng văn bản, nhưng được xây dựng dưới 100 dòng lệnh
- Tuy nhiên, có một hạn chế, đó là cửa sổ soạn thảo có sẵn này chỉ cho phép soạn thảo tối đa 30000 ký tự

Các kiểu lớp Edit

- Văn bản trong các cửa sổ soạn thảo có thể được căn lề theo bên trái, theo bên phải, hoặc ở giữa:
  - ES_LEFT – căn theo bên trái
  - ES_RIGHT – căn theo bên phải
  - ES_CENTER – căn ở giữa
- Mặc định, cửa sổ soạn thảo chỉ có một dòng đơn
- Tạo một cửa sổ soạn thảo đa dòng với kiểu cửa sổ ES_MULTILINE

Các kiểu lớp Edit

- Đối với cửa sổ soạn thảo đơn dòng:
  - Bình thường ta chỉ có thể nhập văn bản đến cuối hình chữ nhật của cửa sổ soạn thảo
  - Để tạo một cửa sổ soạn thảo mà tự động cuốn theo chiều ngang, hãy sử dụng kiểu ES_AUTOHSCROLL
- Đối với cửa sổ soạn thảo đa dòng:
  - Văn bản sẽ tự động xuống dòng trừ khi ta sử dụng kiểu ES_AUTOHSCROLL
  - Nếu sử dụng kiểu ES_AUTOHSCROLL, ta phải nhấn phím Enter để bắt đầu một dòng mới
  - Thêm khả năng cuốn theo chiều dọc bằng việc sử dụng kiểu ES_AUTOVSCROLL

Các kiểu lớp Edit

- Khi các cửa sổ soạn thảo đa dòng đã có các kiểu cuốn, ta vẫn có thể gắn thêm các thanh cuốn vào chúng bằng cách sử dụng các kiểu WS_HSCROLL và WS_VSCROLL (giống như đối với các cửa sổ thông thường)
- Mặc định, một cửa sổ soạn thảo không có viền. Thêm viền bằng việc sử dụng kiểu WS_BORDER
- Kiểu ES_NOHIDESEL
  - Khi lựa chọn văn bản trong một cửa sổ soạn thảo, Windows làm nổi nó trong video đảo
  - Tuy nhiên, khi cửa sổ soạn thảo đánh mất tiêu điểm nhập, văn bản đã lựa chọn không còn được làm nổi nữa
  - Nếu ta muốn vùng lựa chọn vẫn được làm nổi ngay cả khi cửa sổ soạn thảo không sở hữu tiêu điểm nhập, sử dụng kiểu ES_NOHIDESEL

Cửa sổ soạn thảo

- Các cửa sổ soạn thảo gửi các thông điệp WM_COMMAND tới cửa sổ cha

LOWORD (wParam)	Định danh của cửa sổ con
HIWORD (wParam)	Mã thông báo
lParam	Handle tới cửa sổ con

Các mã thông báo

EN_SETFOCUS	Cửa sổ soạn thảo nhận được tiêu điểm nhập
EN_KILLFOCUS	Cửa sổ soạn thảo đánh mất tiêu điểm nhập
EN_CHANGE	Nội dung của cửa sổ soạn thảo thay đổi (gửi sau khi đã cập nhật màn hình)
EN_UPDATE	Nội dung của cửa sổ soạn thảo đã thay đổi (gửi trước khi cập nhật màn hình)
EN_ERRSPACE	Cửa sổ soạn thảo không thể cấp phát đủ bộ nhớ cho một yêu cầu nào đó
EN_MAXTEXT	Cửa sổ soạn thảo đã hết không gian để chèn thêm văn bản (văn bản chèn vào sẽ bị cắt xén)
EN_HSCROLL	Thanh cuốn ngang của cửa sổ vừa được nhấn
EN_VSCROLL	Thanh cuốn dọc của cửa sổ vừa được nhấn

Làm việc với cửa sổ soạn thảo đơn dòng

- Nếu sử dụng vài cửa sổ soạn thảo đơn dòng trên bề mặt của cửa sổ chính:
  - Cần sử dụng kỹ thuật phân lớp cửa sổ để di chuyển tiêu điểm nhập từ cửa sổ này sang cửa sổ khác
  - Các hàm làm việc với văn bản
    - SetWindowText - thiết lập văn bản cho cửa sổ
    - GetWindowTextLength - lấy độ dài văn bản trong cửa sổ
    - GetWindowText - lấy văn bản trong cửa sổ

Làm việc với cửa sổ soạn thảo đơn dòng

- Những thông điệp sau cho phép ta cắt, sao chép, hoặc xoá lựa chọn hiện thời
  - SendMessage (hwndEdit, WM_CUT, 0, 0) ;
  - SendMessage (hwndEdit, WM_COPY, 0, 0) ;
  - SendMessage (hwndEdit, WM_CLEAR, 0, 0) ;
- Chèn văn bản trong clipboard vào cửa sổ soạn thảo tại vị trí con chạy hiện thời:

  SendMessage (hwndEdit, WM_PASTE, 0, 0) ;

Làm việc với cửa sổ soạn thảo đơn dòng

- Lấy các vị trí đầu và cuối của lựa chọn hiện thời:

```
SendMessage (hwndEdit, EM_GETSEL, (WPARAM) &iStart,
    (LPARAM) &iEnd) ;
```

- Lựa chọn văn bản:

```
SendMessage (hwndEdit, EM_SETSEL, iStart, iEnd) ;
```

- Thay thế lựa chọn hiện thời bằng văn bản khác:

```
SendMessage (hwndEdit, EM_REPLACESEL, 0, (LPARAM)
    szString) ;
```

Làm việc với cửa sổ soạn thảo đa dòng

- Lấy số lượng dòng:

```
iCount = SendMessage (hwndEdit, EM_GETLINECOUNT, 0, 0) ;
```

- Lấy vị trí của 1 dòng (tính từ đầu văn bản) trong bộ đệm của cửa sổ soạn thảo:

```
iOffset = SendMessage (hwndEdit, EM_LINEINDEX, iLine, 0) ;
```

- Lấy độ dài của 1 dòng

```
iLength = SendMessage (hwndEdit, EM_LINELENGTH, iLine, 0) ;
```

- Sao chép một dòng vào trong bộ đệm

```
iLength = SendMessage (hwndEdit, EM_GETLINE, iLine,
    (LPARAM)szBuffer) ;
```

Lớp hộp danh sách (listbox)

- Là tập hợp các xâu văn bản được hiển thị dưới dạng một danh sách dạng cột có thể cuốn được bên trong một hình chữ nhật
- Chương trình có thể thêm hoặc bớt các xâu trong danh sách bằng cách gửi các thông điệp tới hộp danh sách
- Gửi các thông điệp WM_COMMAND tới cửa sổ cha của nó khi một mục trong danh sách được chọn

Lớp hộp danh sách

- Các kiểu hộp danh sách:
  - Chỉ cho phép lựa chọn đơn
  - Cho phép lựa chọn kép
- Khi một hộp danh sách sở hữu tiêu điểm nhập
  - Nó hiển thị một đường gạch ngang (con chạy) bao quanh một mục trong hộp danh sách đó
  - **Lưu ý:**
    - Con chạy không biểu thị mục được chọn trong hộp danh sách!!!
    - Mục được chọn được biểu thị bằng việc làm nổi trong video đảo

Hộp danh sách lựa chọn đơn

- Có thể lựa chọn mục mà con chạy đang đứng bằng việc nhấn thanh dài
- Các phím mũi tên
  - Cuốn hộp danh sách
  - Di chuyển con chạy
  - Di chuyển lựa chọn hiện thời
- Các phím Page Up và Page Down
  - Cuốn hộp danh sách
  - Di chuyển con chạy
  - Nhưng không di chuyển lựa chọn

Hộp danh sách lựa chọn đơn

- Việc nhấn một phím chữ cái sẽ di chuyển con chạy và lựa chọn tới mục đầu tiên (hoặc tiếp theo) mà bắt đầu bằng chữ cái đó
- Một mục cũng có thể được lựa chọn bằng việc nháy hoặc nháy kép chuột trên mục này

Hộp danh sách lựa chọn kép

- Phím dài sẽ chuyển đổi trạng thái lựa chọn của mục mà tại đó con chạy đang đứng
- Các phím mũi tên
  - Loại bỏ tất cả các mục được lựa chọn trước đó
  - Di chuyển con chạy
  - Di chuyển lựa chọn
- Phím Ctrl kết hợp với các phím mũi tên
  - Di chuyển con chạy
  - Không hề di chuyển lựa chọn

Hộp danh sách lựa chọn kép

- Phím Shift kết hợp với các phím mũi tên - mở rộng lựa chọn
- Nháy hoặc nháy kép một mục
  - Loại bỏ tất cả các mục đã lựa chọn trước đó
  - Lựa chọn mục đã nháy
- Nháy một mục trong khi nhấn phím Shift:
  - Chuyển đổi trạng thái lựa chọn của mục đó
  - Không hề thay đổi trạng thái lựa chọn của bất kỳ một mục nào khác

Các kiểu hộp danh sách

- Tạo một hộp danh sách bằng CreateWindow với:
  - listbox - tên lớp cửa sổ
  - WS_CHILD - kiểu cửa sổ
- Tuy nhiên, kiểu hộp danh sách mặc định này sẽ không gửi các thông điệp WM_COMMAND tới cửa sổ cha của nó
- Các hộp danh sách gần như luôn sử dụng kiểu LBS_NOTIFY, kiểu này cho phép cửa sổ cha nhận các thông điệp WM_COMMAND từ hộp danh sách
- Nếu muốn hộp danh sách sắp xếp các mục trong nó, sử dụng LBS_SORT

Các kiểu hộp danh sách

- Thêm viền với kiểu cửa sổ WS_BORDER
- Thêm thanh cuốn dọc, sử dụng kiểu WS_VSCROLL
- Các tệp header của Windows định nghĩa kiểu hộp danh sách LBS_STANDARD mà có chứa hầu hết các kiểu thông dụng:

  (LBS_NOTIFY ¦ LBS_SORT ¦ WS_VSCROLL ¦ WS_BORDER)

- Có thể sử dụng các kiểu WS_SIZEBOX và WS_CAPTION, nhưng chúng sẽ cho phép người sử dụng
  - Thay đổi kích thước hộp danh sách
  - Và di chuyển nó xung quanh miền thao tác của cửa sổ cha

Kích thước hộp danh sách

- Chiều rộng tối thiểu của hộp danh sách = chiều rộng của xâu dài nhất + chiều rộng của thanh cuốn
- Lấy chiều rộng của thanh cuốn dọc

```
GetSystemMetrics (SM_CXVSCROLL) ;
```

- Có thể tính toán chiều cao của hộp danh sách = chiều cao của một ký tự × số lượng mục muốn hiển thị

Thêm các xâu vào hộp danh sách

- Các xâu văn bản có chỉ số bắt đầu từ 0 đối với mục cao nhất
- Thêm 1 xâu vào hộp danh sách nhờ hàm SendMessage:
  - Tham số lParam - con trỏ tới xâu được kết thúc bởi ký tự '\0'
  - Hàm trả ra:
    - LB_ERRSPACE - nếu thủ tục cửa sổ đã hết không gian nhớ để lưu trữ nội dung của hộp danh sách
    - LB_ERR - nếu một lỗi xuất hiện đối với các nguyên nhân khác
    - LB_OKAY - nếu thành công

Thêm các xâu vào hộp danh sách

- Nếu hộp danh sách có kiểu LBS_SORT, cách dễ dàng nhất là:

```
SendMessage (hwndList, LB_ADDSTRING, 0, (LPARAM)
    szString) ;
```

- Nếu hộp danh sách không có kiểu LBS_SORT, ta có thể chỉ rõ vị trí cần thêm xâu:

```
SendMessage (hwndList, LB_INSERTSTRING, iIndex,
    (LPARAM) szString) ;
```

  - Nếu iIndex bằng -1 thì xâu mới sẽ được đẩy xuống đáy danh sách

- Có thể dùng LB_INSERTSTRING với các hộp danh sách có kiểu LBS_SORT, nhưng nội dung của hộp danh sách sẽ không được sắp xếp lại

Xoá một xâu từ hộp danh sách

- Xóa 1 xâu trong hộp danh sách:

```
SendMessage (hwndList, LB_DELETESTRING, iIndex, 0) ;
```

- Xoá cả hộp danh sách:

```
SendMessage (hwndList, LB_RESETCONTENT, 0, 0) ;
```

Bật/tắt cờ vẽ lại của hộp danh sách



- Thủ tục cửa sổ của hộp danh sách sẽ cập nhật sự hiển thị mỗi khi:
  - Một mục được thêm vào
  - Hay một mục bị xoá đi
- Nếu cần phải thêm vào hay xoá đi một số xâu:
  - Có thể tạm thời ngăn chặn hành động cập nhật bằng cách tắt cờ vẽ lại trước khi thêm các xâu:

```
SendMessage (hwndList, WM_SETREDRAW, FALSE, 0) ;
```

  - Sau khi hoàn thành việc thêm các xâu, ta có thể bật lại cờ vẽ lại:

```
SendMessage (hwndList, WM_SETREDRAW, TRUE, 0) ;
```

- Hộp danh sách được tạo với kiểu LBS_NOREDRAW sẽ khởi đầu với cờ vẽ lại là tắt



Lựa chọn các mục trong hộp danh sách

- Các lời gọi hàm SendMessage dưới đây:
  - Thông thường có trả ra giá trị
  - Nếu một lỗi xuất hiện, giá trị này được thiết lập là LB_ERR
- Lấy số lượng mục có trong hộp danh sách:

```
iCount = SendMessage (hwndList, LB_GETCOUNT, 0, 0) ;
```

Lựa chọn các mục trong hộp danh sách

- Các hộp danh sách lựa chọn đơn
  - Lựa chọn 1 mục tại 1 vị trí cụ thể

```
SendMessage (hwndList, LB_SETCURSEL, iIndex, 0) ;
```

    - Nếu iIndex = -1 - tất cả các mục đều bị bỏ lựa chọn
  - Lựa chọn một mục dựa trên các ký tự đầu tiên của nó:

```
iIndex = SendMessage (hwndList, LB_SELECTSTRING, iIndex,
    (LPARAM) szSearchString) ;
```

  - Xác định chỉ số của lựa chọn hiện thời:

```
iIndex = SendMessage (hwndList, LB_GETCURSEL, 0, 0) ;
```

Lựa chọn các mục trong hộp danh sách

- Các hộp danh sách lựa chọn đơn
  - Xác định độ dài 1 xâu:

```
iLength = SendMessage (hwndList, LB_GETTEXTLEN, iIndex, 0) ;
```

  - Sao chép một mục vào trong bộ đệm:

```
iLength = SendMessage (hwndList, LB_GETTEXT, iIndex,
    (LPARAM) szBuffer) ;
```

Lựa chọn các mục trong hộp danh sách

- Hộp danh sách lựa chọn kép
  - Không thể sử dụng LB_SETCURSEL, LB_GETCURSEL, hoặc LB_SELECTSTRING
  - Sử dụng LB_SETSEL để thiết lập trạng thái lựa chọn của một mục mà không ảnh hưởng tới các mục khác:

```
SendMessage (hwndList, LB_SETSEL, wParam, iIndex) ;
```

    - wParam ≠ 0 – mục được lựa chọn
    - wParam = 0 – mục bị bỏ lựa chọn
    - iIndex = -1 – tất cả các mục hoặc là được lựa chọn, hoặc là bị bỏ lựa chọn
  - Xác định trạng thái lựa chọn của một mục:

```
iSelect = SendMessage (hwndList, LB_GETSEL, iIndex, 0) ;
```

    - iSelect ≠ 0 – mục đang được lựa chọn
    - iSelect = 0 – mục chưa được lựa chọn

Các thông điệp từ hộp danh sách

- Khi nhấn chuột trên hộp danh sách, hộp danh sách sẽ nhận được tiêu điểm nhập
- Thông điệp WM_COMMAND được gửi từ hộp danh sách tới cửa sổ cha:

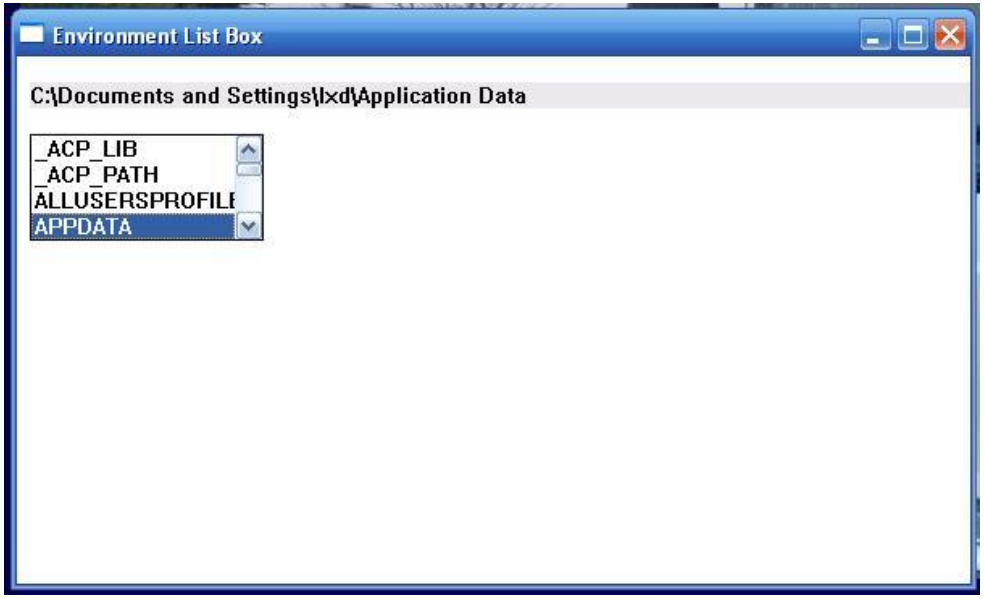
LOWORD (wParam)	Định danh của cửa sổ con
HIWORD (wParam)	Mã thông báo
lParam	Handle tới cửa sổ con

Các thông điệp từ hộp danh sách

Các mã thông báo

LBN_ERRSPACE	Hộp danh sách không còn đủ bộ nhớ để thực hiện yêu cầu
LBN_SELCHANGE	Chuẩn bị thay đổi lựa chọn trong hộp danh sách
LBN_DBLCLK	Người sử dụng nháy kép chuột lên 1 xâu trong hộp danh sách
LBN_SELCANCEL	Người sử dụng hủy bỏ lựa chọn trong hộp danh sách
LBN_SETFOCUS	Hộp danh sách nhận tiêu điểm nhập
LBN_KILLFOCUS	Hộp danh sách mất tiêu điểm nhập

Chương trình Environ



- Hiển thị tên các biến môi trường hiện thời của hệ điều hành (như là PATH và WINDIR)
- Khi lựa chọn một biến môi trường, xâu môi trường sẽ được hiển thị ở trên đỉnh của miền thao tác

Lớp thanh cuốn

- Cần phân biệt sự khác nhau giữa:
  - Các thanh cuốn của các cửa sổ - xuất hiện ở bên phải và dưới đáy của cửa sổ
  - Và các thanh cuốn điều khiển - là các cửa sổ con mà có thể xuất hiện ở bất cứ đâu bên trong miền thao tác của cửa sổ cha
- Tạo các thanh cuốn điều khiển bằng cách sử dụng lớp cửa sổ scrollbar
- Có 2 kiểu thanh cuốn:
  - SBS_VERT
  - SBS_HORZ

Lớp thanh cuốn

- Các thanh cuốn điều khiển
  - Không gửi các thông điệp COMMAND
  - Mà gửi các thông điệp WM_VSCROLL và WM_HSCROLL tới cửa sổ cha
- Các thông điệp thanh cuốn:
  - Tham số lParam bằng:
    - 0 - đối với các thanh cuốn của các cửa sổ
    - Handle của cửa sổ - đối với các thanh cuốn điều khiển
  - Các từ cao và thấp của tham số wParam có ý nghĩa giống nhau đối với 2 loại thông điệp thanh cuốn:
    - LOWORD(wParam): Lấy ra các mã thông báo (thông điệp) trên thanh cuốn
    - HIWORD(wParam): Lấy ra vị trí của thanh cuốn khi rê chuột.

Lớp thanh cuốn

- Các thanh cuốn điều khiển cũng có thể xử lý các sự kiện nhấn phím, nhưng chỉ khi chúng đang sở hữu tiêu điểm nhập

Phím dịch chuyển con chạy	Giá trị wParam trong thông điệp thanh cuốn
Home	SB_TOP
End	SB_BOTTOM
Page Up	SB_PAGEUP
Page Down	SB_PAGEDOWN
Left hoặc Up	SB_LINEUP
Right hoặc Down	SB_LINEDOWN

Lớp thanh cuốn

- Các thanh cuốn của các cửa sổ có độ rộng cố định
- Nhưng các thanh cuốn điều khiển có thể có kích thước tuỳ ý
- Thiết lập phạm vi và vị trí của thanh cuốn điều khiển:

```
SetScrollRange (hwndScroll, SB_CTL, iMin, iMax, bRedraw) ;
SetScrollPos (hwndScroll, SB_CTL, iPos, bRedraw) ;
SetScrollInfo (hwndScroll, SB_CTL, &si, bRedraw) ;
```

Các màu hệ thống

- Windows duy trì 29 màu hệ thống để tô vẽ các thành phần khác nhau trên màn hình
- Ta có thể biết và thiết lập các màu này bằng cách sử dụng
  - GetSysColor
  - SetSysColors - chỉ làm thay đổi màu trong phiên (session) hiện thời của Windows!!!

Các màu hệ thống

- Ta cũng có thể thay đổi một số (nhưng không phải tất cả) màu hệ thống bằng cách sử dụng mục Display trong Control Panel của Windows
- Các màu đã chọn được lưu trữ trong:
  - Registry đối với Microsoft Windows NT
  - Tệp WIN.INI đối với Windows 98

Giải quyết vấn đề ở trên

- Sử dụng COLOR_BTNFACE cho nền của miền thao tác khi ta định nghĩa lớp cửa sổ:

```
wndclass.hbrBackground = (HBRUSH)(COLOR_BTNFACE + 1) ;
```

- Nhưng một vấn đề khác nảy sinh khi ta hiển thị văn bản thông qua hàm TextOut:
  - Windows sẽ dùng các giá trị được định nghĩa trong ngữ cảnh thiết bị để làm màu nền và màu văn bản
  - Các giá trị mặc định:
    - Màu nền - trắng ⇒ KHÔNG HỢP LÝ
    - Màu văn bản – đen

Giải quyết vấn đề ở trên

- Thay đổi màu nền và màu văn bản giống như các màu hệ thống:

```
SetBkColor (hdc, GetSysColor (COLOR_BTNFACE)) ;
SetTextColor (hdc, GetSysColor (COLOR_WINDOWTEXT)) ;
```

- Để thay đổi màu chữ (màu nền) của các loại cửa sổ điều khiển ta cần xử lý thông điệp WM_CTLCOLOR…
  - WM_CTLCOLORSCROLLBAR
  - WM_CTLCOLORSTATIC
  - …
  - Và trả về một đối tượng (LRESULT)HBRUSH

Giải quyết vấn đề ở trên

- Nếu người sử dụng thay đổi các màu hệ thống trong khi chương trình đang chạy, ta cần phải cập nhật lại màu cho các cửa sổ như sau:

```
case WM_SYSCOLORCHANGE:
    InvalidateRect (hwnd, NULL, TRUE) ;
    break ;
```

Chương trình COLORS2

